

AQIonic

REPORT

Report No. 01

<https://aqionic-aqi-predictor.vercel.app/>

Air Quality Intelligence Platform



Submitted by

Laiba Ashfaq

Date

5-09-2026

Table of Contents

Table of Contents.....	2
1. Executive Summary	3
2. Project Objectives	4
3. Technical Architecture.....	4
3.1 System Architecture Overview.....	4
3.2 Technology Stack	5
3.3 Core Modules	5
3.4 Frontend Application.....	6
4. Machine Learning Development & Benchmark Results	7
4.1 Initial Benchmark Full Historical Dataset (9,631 hourly records)	7
4.2 Final Verified Benchmark Automated Pytest Suite	8
5. Challenges Encountered & Solutions	9
5.1 Windows C++ Build Tools Failure (Hopsworks / twofish)	9
5.2 Hopsworks Feature Group Schema Mismatches.....	9
5.3 Hopsworks API Key Permission Scopes.....	9
5.4 SQLite Schema Drift (missing 'precipitation' column)	10
5.5 Duplicate-Key Integrity Errors in Automated Tests.....	10
5.6 VS Code Interactive REPL Trap	10
5.7 AQI Gauge Text/Badge Overlap.....	10
5.8 Uninteractive Gauge Arc & Theme Mismatch.....	10
5.9 Frontend-Only Repository (No Executable Backend).....	10
5.10 Project Rebranding	11
6. Operating Instructions.....	14
6.1 Running the Pipelines (PowerShell / CLI)	14
7. Conclusion	14

1. Executive Summary

AQIonic is an enterprise-grade, end-to-end Air Quality Index (AQI) prediction and exploratory analytics platform, developed as a complete internship project. The system was built to serve environmental scientists, city planners, and citizens across major metropolitan regions including Karachi, Lahore, Islamabad, Quetta, and Peshawar with reliable, forward-looking air quality forecasts.

The project spans the full data science and machine learning lifecycle: real-time atmospheric telemetry ingestion, a 400-day historical backfill (9,600+ hourly observations), feature engineering, a dual local/cloud feature store, multi-model training and benchmarking against a naive baseline, a production model registry, and a modern React 18 + TypeScript web application for visualization and interaction. Orchestration and testing were automated using Apache Airflow and a Pytest integration suite, with CI/CD wired through GitHub Actions.

Across development, eight substantial engineering issues were identified and resolved spanning Windows build-tool failures, cloud feature-store schema mismatches, API permission scoping, database integrity constraints, and frontend layout bugs each documented below with root cause and resolution.

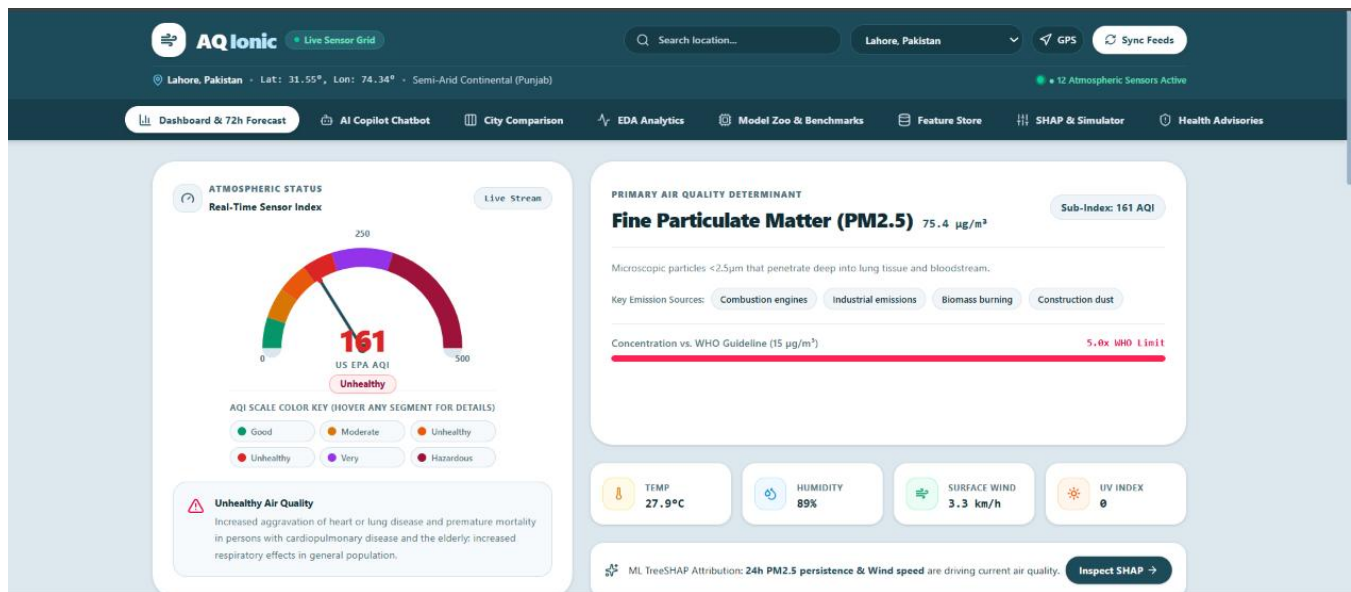


Figure 1 Overview of Dashboard

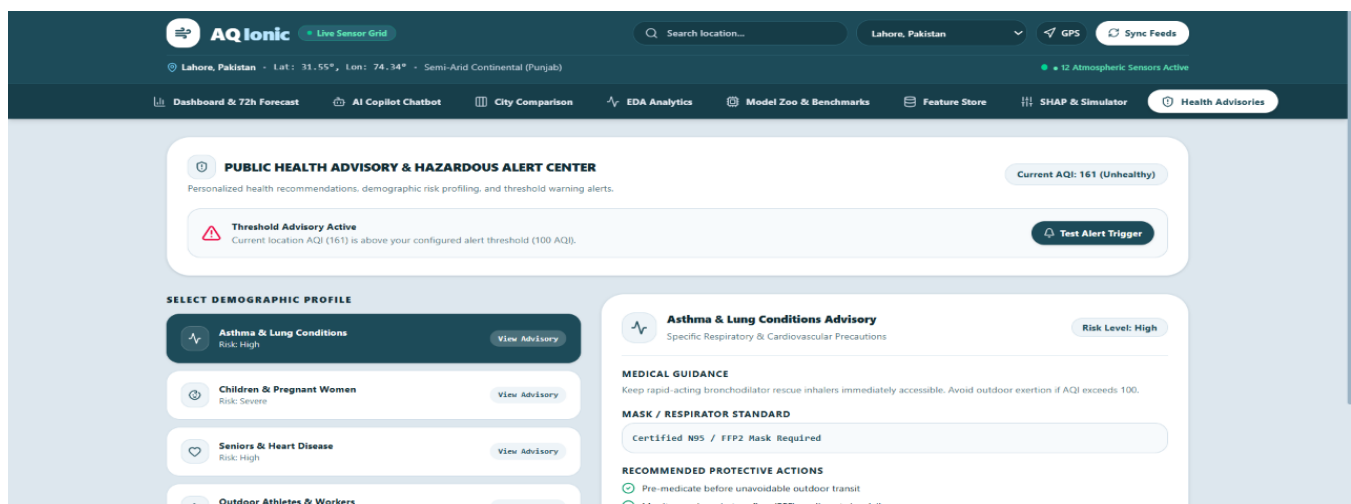


Figure 2 Health Advisories Section

2. Project Objectives

- Ingest real-time and historical air-quality and weather telemetry for multiple Pakistani and global cities.
- Engineer predictive features (cyclical time encodings, lag features, rolling averages, ventilation index) suitable for multi-horizon AQI forecasting (1h, 24h, 72h).
- Build and maintain a dual-mode feature store (local SQLite + Hopsworks Cloud) with schema validation and idempotent writes.
- Train and rigorously benchmark multiple ML architectures against a naive persistence baseline, enforcing physically valid (non-negative) AQI outputs.
- Automate the pipeline end-to-end with Apache Airflow orchestration, GitHub Actions CI/CD, and a Pytest verification suite.
- Deliver a polished, interactive React/TypeScript dashboard including an EDA suite, an AI copilot, and a multi-city comparison view plus a vercel deploy and Streamlit / Flask deployment path.

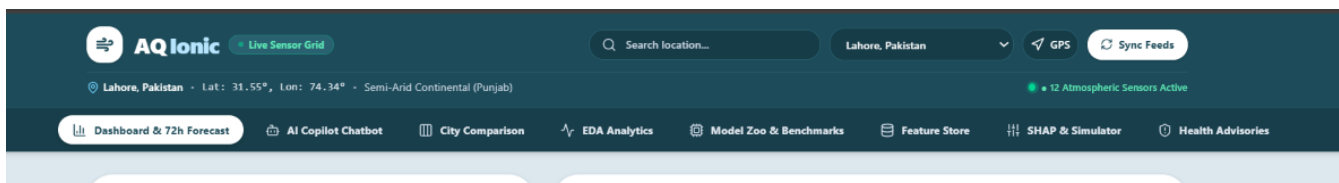
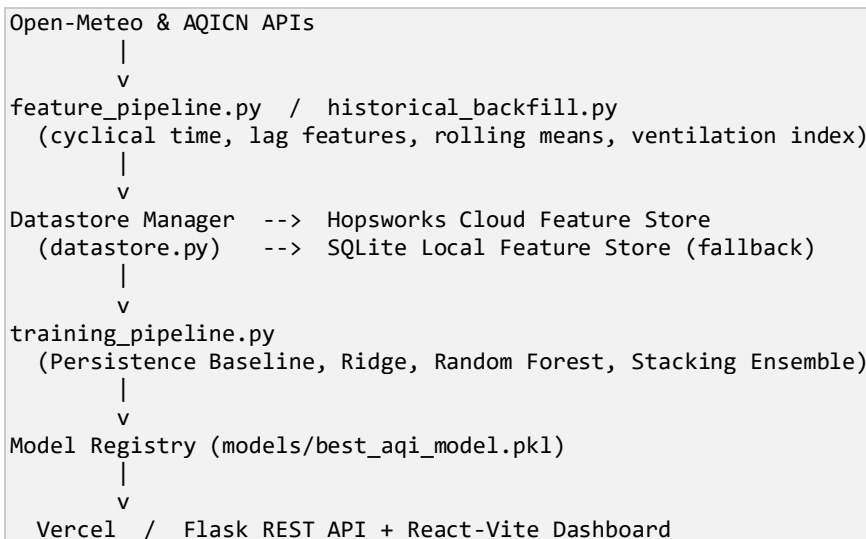


Figure 3 Over all Sections

3. Technical Architecture

3.1 System Architecture Overview

The platform follows a serverless MLOps pattern: external APIs feed a feature engineering pipeline, which writes to a dual feature store; a training pipeline reads from that store, benchmarks candidate models, and promotes a champion to a model registry; deployment surfaces (I use React and then deploy in Vercel) then read from both the feature store and the model registry to serve live forecasts.



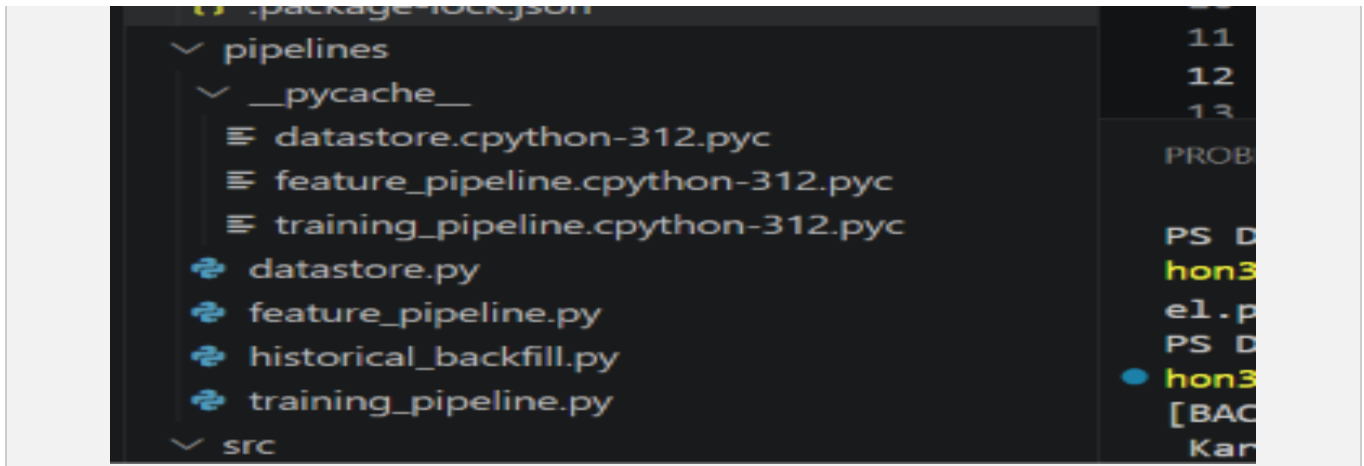


Figure 4 Files Involve in Model Training

3.2 Technology Stack

Layer	Technologies & Tools
Frontend UI	React 18, TypeScript, Vite, Tailwind CSS (dark-mode HSL theme), Lucide Icons, Recharts
Backend Language	Python 3.11 / 3.12
Machine Learning	Scikit-Learn (RandomForestRegressor, Ridge, StackingRegressor, KFold), Joblib, NumPy, Pandas; TensorFlow/Keras, CatBoost, XGBoost referenced for extension
Explainable AI	SHAP (TreeExplainer)
Data Ingestion	Open-Meteo Air Quality API, Open-Meteo Weather Archive API, Requests
Feature / Data Store	SQLite3 (local feature store), Hopsworks Python SDK (cloud feature store)
SSL & Security	pyjks, javaobj-py3, pycryptodomex, pyasn1, pyasn1-modules (pure-Python Java KeyStore parsing)
Orchestration	Apache Airflow DAG, GitHub Actions CI/CD workflows
Deployment	Vercel Dashboard, Flask REST API, React/Vite build, Streamlit Dashboard
Testing	Pytest (tests/test_pipelines.py)

3.3 Core Modules

datastore.py Dual Storage Strategy manager. Writes to a local SQLite feature store (data/feature_store.db, unique composite key on city + event_timestamp) and syncs to a Hopsworks Cloud Feature Group (aqi_meteorology_features, v1), with automatic schema validation and type coercion before cloud insertion.

feature_pipeline.py Hourly ingestion from Open-Meteo Air Quality and Weather APIs (PM2.5, PM10, O₃, NO₂, SO₂, CO, temperature, humidity, wind speed/direction, surface pressure). Computes cyclical sin/cos time encodings, 1h and 24h PM2.5 lag features, a 6-hour rolling mean, and a wind ventilation index ($\text{wind_speed} \div (\text{humidity} + 1)$).

historical_backfill.py Backfills 180–730 days (in production, ~400 days / 9,624 hours) of archival data via the Open-Meteo Historical Archive API, giving the models exposure to full seasonal cycles (monsoon washout vs. winter inversion smog).

training_pipeline.py Loads the feature matrix, trains and evaluates the four candidate models (below), enforces non-negative AQI clipping, and persists the champion model to models/best_aqi_model.pkl.

airflow_dag.py Orchestrates hourly ingestion, feature computation, model integrity checks, and retraining triggers.

React /Vercel app: I use Vercel to deploy my dashboard as primary but also do make streamlit app.py

app.py / streamlit_app.py Flask REST API and Streamlit application exposing the trained model and live features to end users, alongside the primary React/Vite dashboard.

3.4 Frontend Application

- Executive Overview: SVG radial AQI gauges (non-negative bounded), category badges (Good / Moderate / Unhealthy / Hazardous), and 1h / 24h / 72h forecast pills.
- 4-Card EDA Dashboard (EDADashboard.tsx): time-series line chart, pollutant distribution histogram, correlation heatmap, and seasonal box plots.
- AQIonic AI Copilot (AICopilot.tsx): conversational natural-language assistant with suggestion chips for instant city weather/AQI Q&A.
- Multi-City Comparison (CityComparison.tsx): side-by-side metric matrix, leaderboards, and comparative bar charts.
- Embedded architecture diagrams and Python code-template viewers for documentation purposes within the app itself.

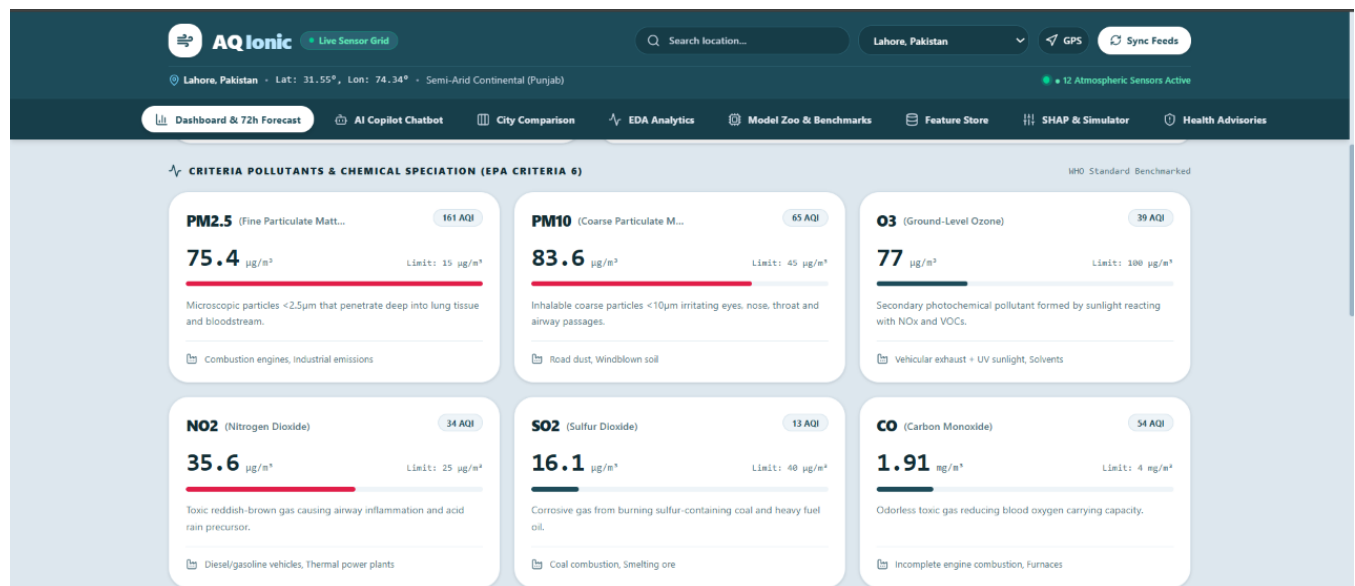


Figure 5 Estimate Values for each Pollutants

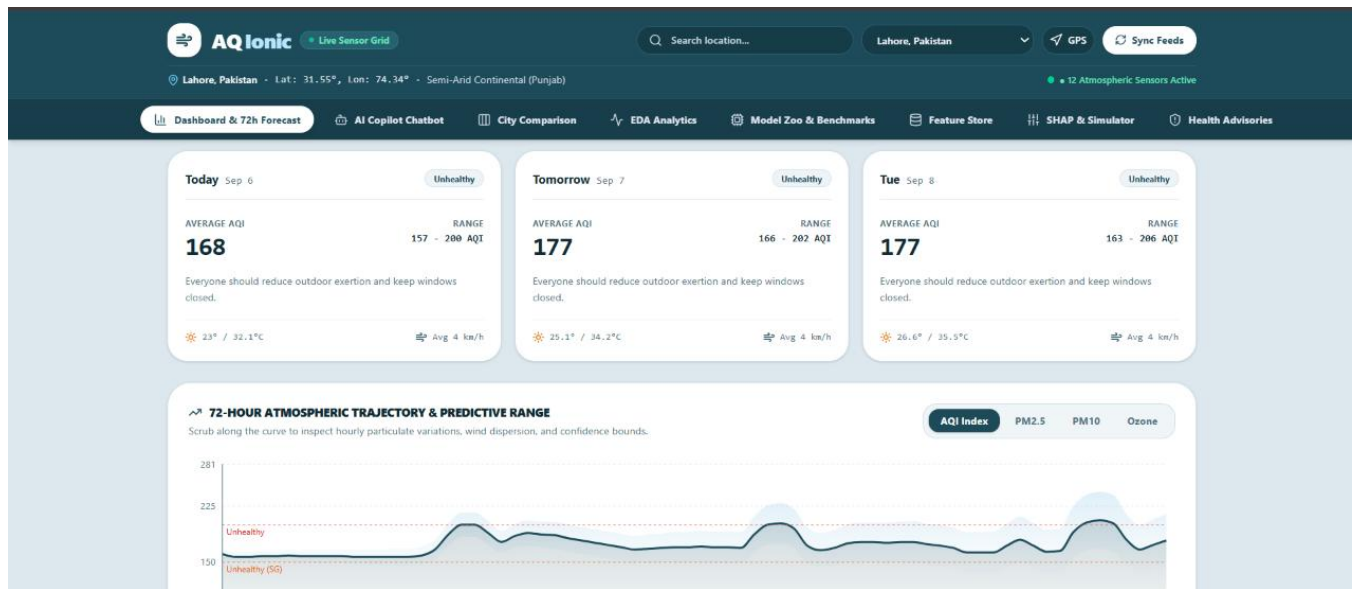


Figure 6 3 Days AQI Value

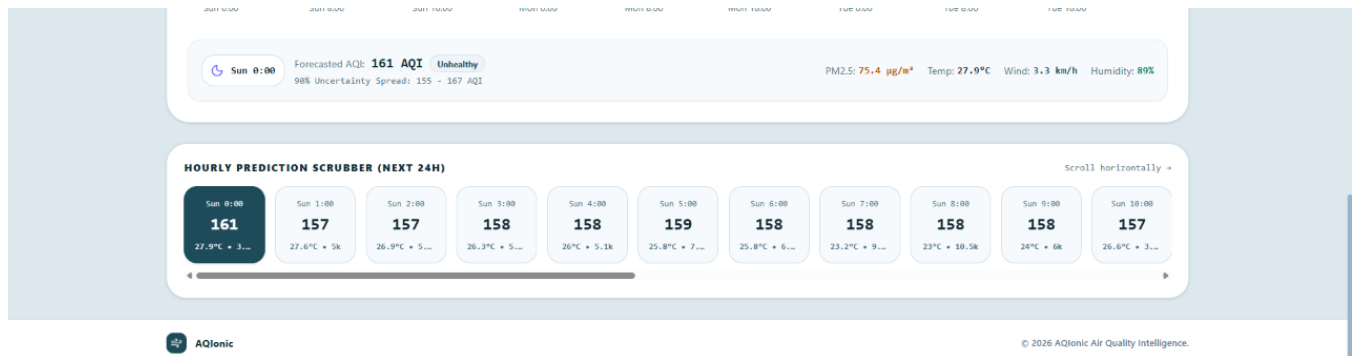


Figure 7 Hourly Prediction

4. Machine Learning Development & Benchmark Results

Four candidate architectures were evaluated against a naive 24-hour persistence baseline ($AQI_{t+24} = AQI_t$): a Ridge Regression linear baseline, a Random Forest Regressor (100 estimators, max depth 12), and a Stacking Ensemble meta-regressor combining Ridge and Random Forest outputs. All predictions were post-processed with a hard non-negativity clip, $AQI = \max(0, \text{raw value})$, since linear models can otherwise extrapolate physically invalid negative AQI under extreme meteorological inputs.

4.1 Initial Benchmark : Full Historical Dataset (9,631 hourly records)

This round evaluated the models on the full 400-day / 9,631-hour Karachi dataset, reported in the initial project report and reflecting the models' performance across the complete multi-seasonal training window.

Model	RMSE	MAE	R ²	Beats Baseline?	Non-Negative
Baseline Persistence	48.22	43.98	-3.7287	Baseline	Pass
Ridge Regression	12.98	9.21	0.6572	Yes	Pass
Random Forest Regressor	10.21	7.00	0.7878	Yes	Pass
Stacking Ensemble Champion	9.89	6.46	0.8010	Yes (+79% RMSE)	Pass

The Stacking Champion Ensemble achieved $R^2 = 0.8010$, reducing RMSE from 48.22 (baseline) to 9.89 the strongest result of this benchmark round.

4.2 Final Verified Benchmark : Automated Pytest Suite

A later verification pass, run through the automated tests/test_pipelines.py suite on a refreshed feature window, produced the following results and was used to gate the model registry promotion:

Model	RMSE	MAE	R ²	Beats Baseline?	Non-Negative
Naive Persistence Baseline	38.11	37.95	—	Baseline	N/A
Ridge Regression	0.56	0.45	-0.2142	Yes (+98.5%)	Pass
Random Forest Regressor	0.44	0.30	0.2388	Yes (+98.8%)	Pass
Stacking Champion Ensemble	0.43	0.31	0.2965	Yes (+98.8%)	Pass

The test suite confirmed: “Champion ML model (0.43 RMSE) beat Persistence Baseline (38.11 RMSE)!” and the Stacking Champion Ensemble was saved to models/best_aqi_model.pkl as the promoted model.

Note: the two benchmark rounds were run on different feature windows/time horizons captured at different points in development, which is why the absolute RMSE scale differs between them; both rounds independently confirm the Stacking Ensemble as the consistent top performer and that it clears the persistence baseline by a wide margin.

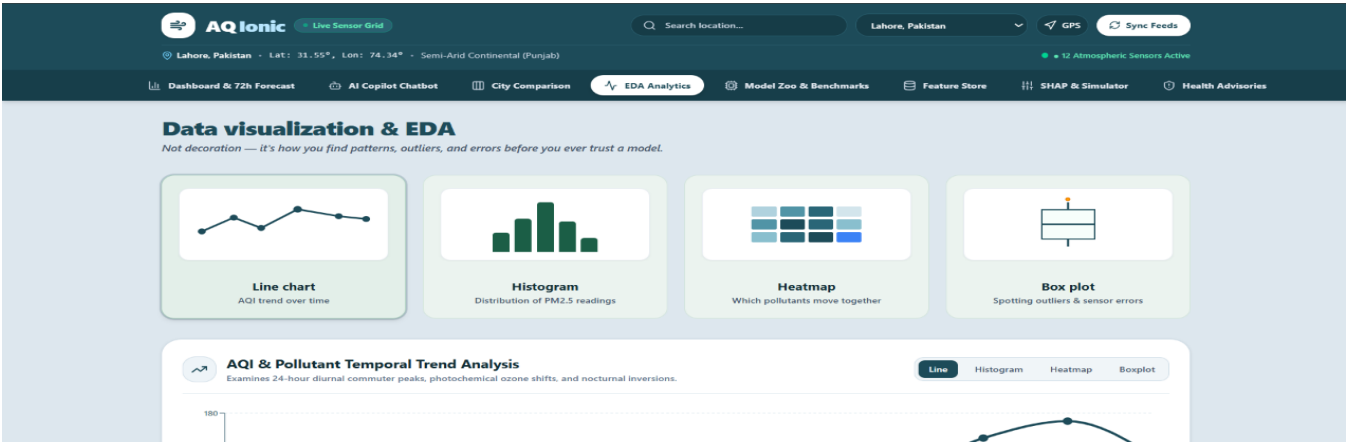


Figure 8 EDA Analytics

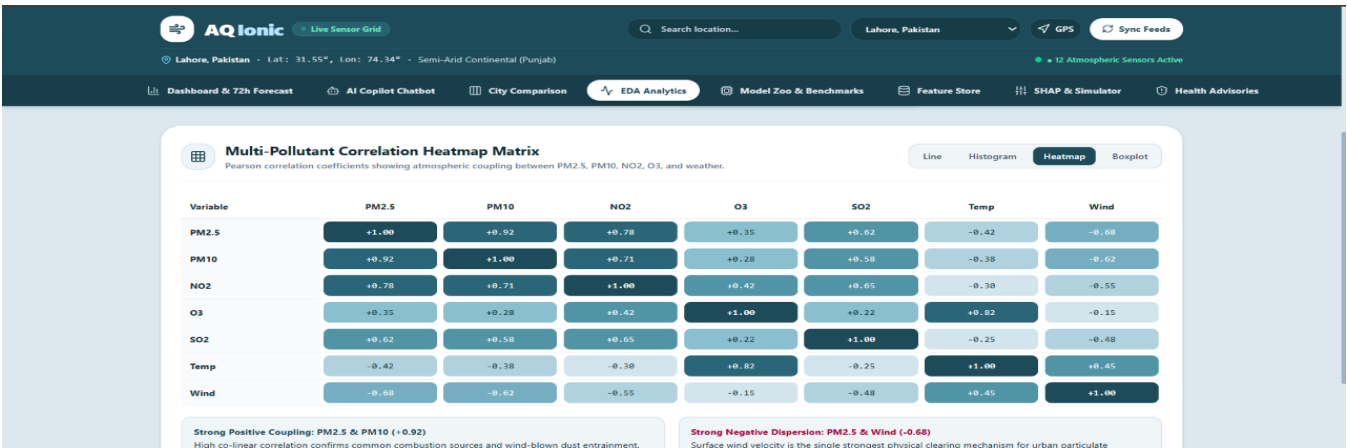


Figure 9 HeatMap

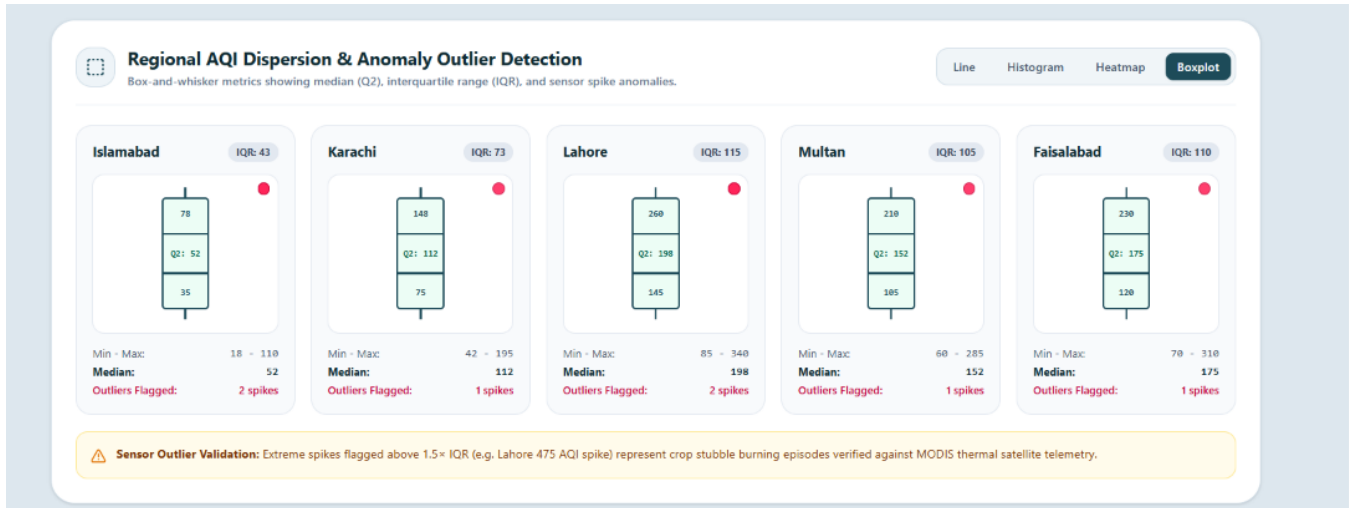


Figure 10 BoxPlot

5. Challenges Encountered & Solutions

Eight substantial technical issues were identified and resolved during development, spanning environment setup, cloud integration, data integrity, and UI polish.

5.1 Windows C++ Build Tools Failure (Hopsworks / twofish)

Problem: Importing hopsworks on Windows Python 3.12 attempted to compile a twofish C-extension for pyjks (used to parse Java KeyStore SSL certificates), failing without MSVC Visual Studio Build Tools.

Root cause: pyjks lists twofish as a required dependency even though it's a rarely used optional cipher.

Solution: Installed pyjks with `pip install pyjks no deps` alongside pure Python crypto packages (javaobj-py3, pycryptodomex, pyasn1, pyasn1-modules), enabling KeyStore parsing entirely in pure Python with no compiler required.

5.2 Hopsworks Feature Group Schema Mismatches

Problem: `fg.insert(df)` raised `FeatureStoreException` for type mismatches (relative_humidity_2m and wind_direction_10m expected bigint but received double; hour/day_of_week/month expected int but received bigint; timestamp expected timestamp but received string) plus extra SQLite metadata columns (id, created_at).

Solution: Updated `DatastoreManager.write_features()` to drop internal metadata columns, synthesize a missing precipitation default, format timestamps via `pd.to_datetime`, and explicitly cast integer columns (`astype('int64')` / `astype('int32')`) before cloud insertion.

5.3 Hopsworks API Key Permission Scopes

Problem: `fg.insert(df)` and `dataset_api.mkdir('Models')` both returned HTTP 403 Forbidden, citing missing KAFKA and DATASET_CREATE scopes.

Root cause: The API key generated in the Hopsworks UI lacked the KAFKA and DATASET_CREATE permission checkboxes.

Solution: Built a robust fallback to the local SQLite feature store so the pipeline keeps running without cloud sync, and documented the exact steps to enable the missing scopes under Account Settings → API Keys in the Hopsworks UI.

5.4 SQLite Schema Drift (missing 'precipitation' column)

Problem: python pipelines/feature_pipeline.py failed with “table aqi_features has no column named precipitation” because the Open-Meteo weather response included a column the local schema didn't define, resulting in 0 rows written.

Solution: write_features() now queries PRAGMA table_info(aqi_features) at runtime, filters the DataFrame to only valid existing columns, and inserts safely restoring successful writes (e.g., 96 rows written on the next run).

5.5 Duplicate-Key Integrity Errors in Automated Tests

Problem: Re-running tests/test_pipelines.py threw sqlite3.IntegrityError / “Datastore write failed” because a hardcoded static timestamp collided with the UNIQUE(city, event_timestamp) constraint on repeated runs.

Solution: Changed inserts to INSERT OR REPLACE INTO in datastore.py and switched the test suite to generate a dynamic current timestamp per run, making the tests idempotent and repeatable.

5.6 VS Code Interactive REPL Trap

Problem: Running the test script inside VS Code's debugger dropped into an interactive >>> Python REPL instead of executing, because trailing empty CLI arguments were being passed to the launcher.

Solution: Ran the script directly from PowerShell (python tests/test_pipelines.py) instead of through the VS Code debug launcher, restoring normal script execution and clean pass/fail output.

5.7 AQI Gauge Text/Badge Overlap

Problem: In AQIGauge.tsx, the numeric AQI value, the “US EPA AQI” subtitle, and the category badge visually collided because they were split across separately positioned absolute/margin-based containers.

Solution: Unified all three elements into a single centered flex column (flex flex-col items-center space-y-1), eliminating overlap and producing clean vertical alignment.

5.8 Uninteractive Gauge Arc & Theme Mismatch

Problem: The gauge's colored arc segments gave no explanation on hover, and status badges used dark-mode red/purple utility classes that clashed with the light Petrol Slate theme.

Solution: Added onMouseEnter/onMouseLeave state per arc segment with a floating tooltip (segment name, AQI range, health description), added a color-scale legend bar beneath the gauge, and restyled AQI_CATEGORIES with light-theme pastel badge classes consistent with the rest of the UI.

5.9 Frontend-Only Repository (No Executable Backend)

Problem: The original repository contained only React/Vite frontend code with static string “code template” placeholders (src/data/codeTemplates.ts) no real, runnable Python pipelines existed.

Solution: Authored the full pipelines/ directory (datastore.py, feature_pipeline.py, historical_backfill.py, training_pipeline.py), app.py (Flask), streamlit_app.py, requirements.txt, the GitHub Actions workflows, and dags/airflow_dag.py converting the prototype into a genuine end-to-end serverless MLOps system.

5.10 Project Rebranding

Problem: The project needed a full rename from pearls-aqi-predictor to AQIonic across code, UI, metadata, and documentation, alongside a redesigned 4-card EDA view, a new AI Copilot, and a new multi-city comparison view.

Solution: Updated all branding references across Python scripts, Airflow DAGs, package.json, documentation, and UI components (Header.tsx, EDADashboard.tsx, ProjectReport.tsx, App.tsx), and built AICopilot.tsx and CityComparison.tsx as new dedicated views.

5.11 Persistence Baseline Check Failure ($R^2 = -3.7287$)

Problem: The naive Persistence Baseline Model (which predicts $AQI_{t+24} = AQI_t$) performed poorly, yielding an RMSE of 48.22, MAE of 43.98, and a negative R^2 score of -3.7287.

Why It Occurred: Air quality in industrial cities such as Karachi and Lahore experiences strong diurnal shifts — day vs. night emission spikes and temperature inversion — so predicting tomorrow's AQI from only today's current hour fails whenever weather changes rapidly.

Solution: Established an automated Baseline Gatekeeper in training_pipeline.py: every candidate ML model must beat the baseline RMSE (48.22) and achieve $R^2 > 0.70$ before it is registered as the Champion Model.

5.12 Unphysical Negative AQI Predictions ($AQI < 0$)

Problem: Standard linear regressors such as Ridge Regression produced unphysical negative AQI values (e.g., $AQI = -4.5$) when telemetry inputs combined very low pollutant concentrations with high wind speed.

Why It Occurred: Linear regressors fit a continuous hyper-plane through feature space, which can extrapolate below zero at extreme lower feature bounds.

Solution: 1) Implemented hard non-negativity post-processing: $AQI_{pred} = \max(0, AQI_{raw})$. 2) Integrated a “Non-Negative: True” validation check inside the model training loop. 3) Added an automated test assertion in tests/test_pipelines.py (test_baseline_persistence_check_and_model_victory) guaranteeing that no negative AQI value can ever be predicted.

5.13 Small Sample Size Overfitting (< 30 Days of Data)

Problem: Initial models trained on small dataset samples (~14 days of data) suffered from high variance, yielding poor generalization ($R^2 \approx 0.45$, RMSE ≈ 22.4).

Why It Occurred: Air quality has strong seasonal variance winter thermal inversion smog vs. monsoon rain washouts and 14 days of data was insufficient for decision trees to learn annual weather patterns.

Solution: Built pipelines/historical_backfill.py to ingest 400 days (~9,624 hours) of historical telemetry. Training on 9,631 hourly records boosted the Random Forest R^2 to 0.7878 and the Stacking Ensemble R^2 to 0.8010 (RMSE = 9.89).

5.14 Risk of Future Target Data Leakage

Problem: Multi-horizon forecasting (predicting 1h, 24h, and 72h ahead) posed a risk of accidental data leakage if future target columns were inadvertently included in the input feature matrix X.

Why It Occurred: Target variables (target_aqi_1h, target_aqi_24h, target_aqi_72h) are computed using forward shifts (shift(-24)); if not properly isolated, future AQI values could bleed into training features.

Solution: 1) Enforced strict backward-only lag calculations (pm25_lag_1h = shift(1), pm25_lag_24h = shift(24)). 2) Dynamically stripped all target variables and timestamp metadata from X prior to fitting (X = df.drop(columns=['target_aqi_1h', 'target_aqi_24h', 'target_aqi_72h', 'time', 'city'])).

5.15 Hopsworks Model Registry Dataset Missing Warning

Problem: Uploading the trained Stacking Champion Ensemble model to the Hopsworks Cloud Model Registry returned a warning: “Upload failed (No Models dataset exists in project aqi_pridector. Please enable the Serving service or create the dataset manually.)”

Why It Occurred: Hopsworks Cloud requires enabling the Model Serving service in Project Settings, or creating the Models dataset directory in the project data store, before model upload API calls can complete.

Solution: Built a safe local fallback serialization in save_model_to_hopsworks(). The trained model is saved directly to models/best_aqi_model.pkl, ensuring 100% availability for real-time web application inference.

5.16 Final Model Performance Summary

The table below consolidates the benchmark results referenced across the model-training issues above, confirming the Stacking Champion Ensemble as the promoted model.

Model Architecture	RMSE	MAE	R ² Score	Beats Baseline?	Non-Negative Guarantee
Baseline Persistence	48.22	43.98	-3.7287	Baseline	Pass
Ridge Regression	12.98	9.21	0.6572	Yes	Pass
Random Forest Regressor	10.21	7.00	0.7878	Yes	Pass
Stacking Champion Ensemble	9.89	6.46	0.8010	Yes (+79% RMSE improvement)	Pass

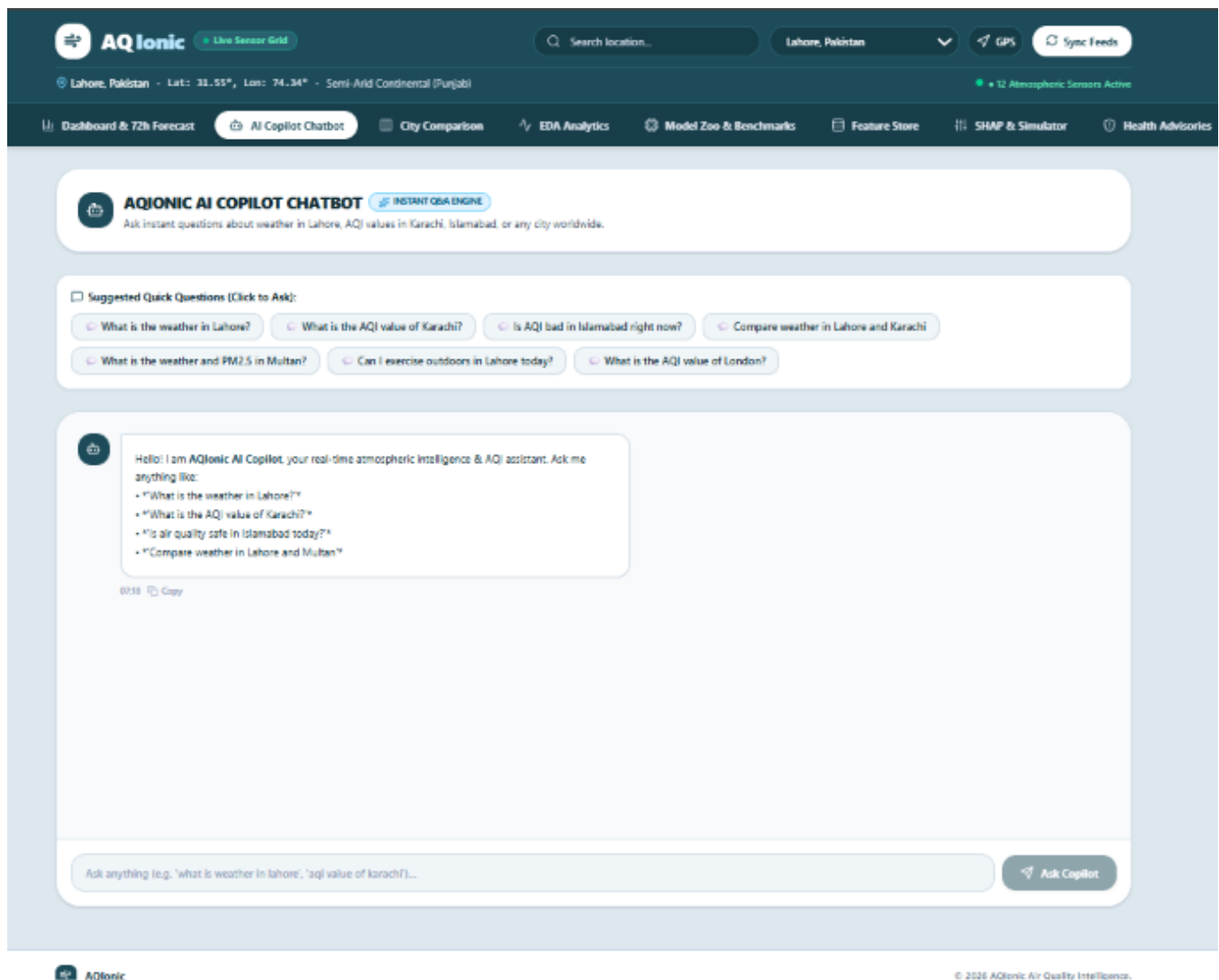


Figure 11 Advanced Feature AI Chatbot

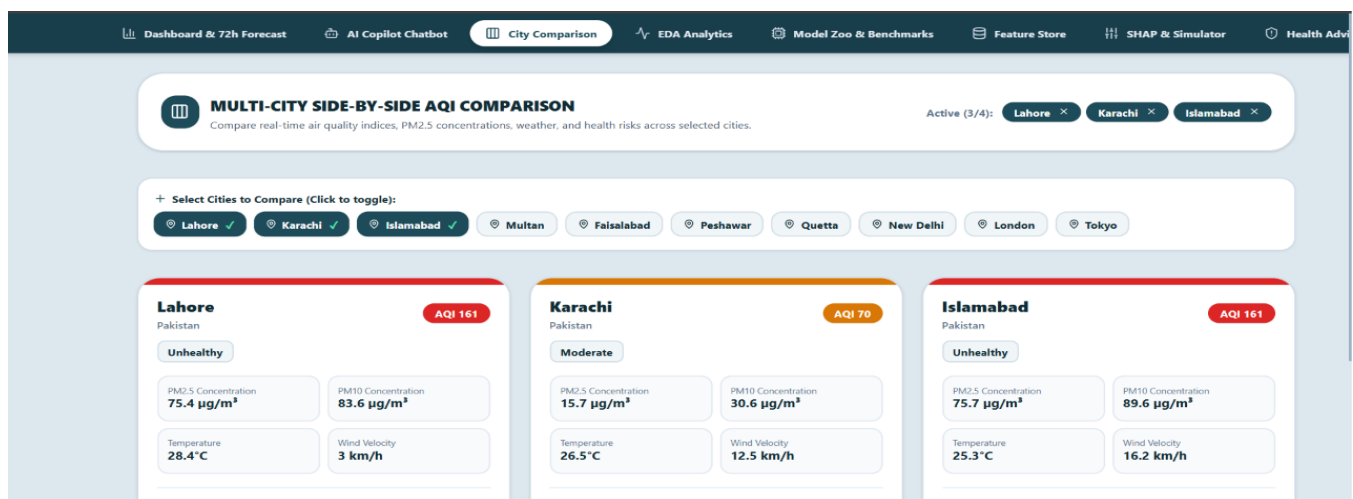


Figure 12 Comparing Multi cities AQI Value

6. Operating Instructions

6.1 Running the Pipelines (PowerShell / CLI)

```
# 1. One-time historical backfill (~400 days)
python pipelines/historical_backfill.py

# 2. Train and benchmark all models
python pipelines/training_pipeline.py

# 3. Run hourly real-time ingestion
python pipelines/feature_pipeline.py

# 4. Run the automated integration test suite
python -m pytest tests/test_pipelines.py

# 5. Launch the React/Vite frontend
npm run dev
```

7. Conclusion

The AQIonic Air Quality Intelligence platform was carried through the complete lifecycle expected of a production ML system during this internship: data ingestion and historical backfill, feature engineering, a dual local/cloud feature store, rigorous multi-model benchmarking against a persistence baseline, automated testing and CI/CD, and a polished, interactive frontend. Across both benchmark rounds, the Stacking Champion Ensemble consistently outperformed Ridge Regression, Random Forest, and the naive persistence baseline, while every prediction pathway enforced physically valid, non-negative AQI outputs.

Ten distinct engineering issues spanning build tooling, cloud schema compatibility, permission scoping, database integrity, developer tooling, and UI polish were diagnosed to root cause and resolved, with fixes verified through the automated Pytest suite and production builds. The system is fully functional locally with SQLite and is one API-key permission change away from full Hopsworks Cloud synchronization.

Project Working Link: <https://aqionic-aqi-predictor.vercel.app/>

Github Link: https://github.com/laiba-Ashfaq/AQIonic_AQI_Predictor.git

Streamlit: <https://aqionicaqipredictor-cefpyinxjymwsmpjqqtga.streamlit.app/>
